共用單畫面與 Bridge UART 更新指南

本指南整理 Tool 的共用畫面，以及 MCU CAN host 的 UART App 更新與工程設定。

1. 一個畫面，三個工作區

畫面以1366×768為最低配置：上方識別卡、左側介面設定、中間共用BIN / metadata / checksum、右側共用log、底部progress / elapsed。 主視窗沒有上下捲軸，log歷史與下拉清單可捲動。 五張卡於小畫面同列、大畫面分兩列，hover可讀全文。

BIN、檔案路徑、18個metadata欄位與log由各TAB共用。

CAN 與 I2C TAB 提供 port、speed、ID / mask 或 slave address 等介面參數及適用的 image contract。 最後一個 TAB 是 Misc，更新目的地為 host bridge。

CAN / I2C / Misc TAB 位於左側 INTERFACE SETTINGS。 中欄 SHARED IMAGE / METADATA 與右欄 SHARED LOG 固定獨立，切換 TAB 時保留共用資料。

顏色與文字	意義
綠色 READY / MATCH	條件符合或資料完整性相符，不代表身分認證
藍色 OFFLINE / WORKING	本地 BIN 資訊或執行中
黃色 REQUIRED / REFERENCE	等待必要條件或僅為範例回報資訊
紅色 ERROR / BLOCKED	錯誤、CRC 不符或不允許操作
灰色 N/A	尚未取得或該協定未提供；離線 BIN 資料與 live 資訊分開顯示

2. Host、Device 與 BIN 的資訊來源

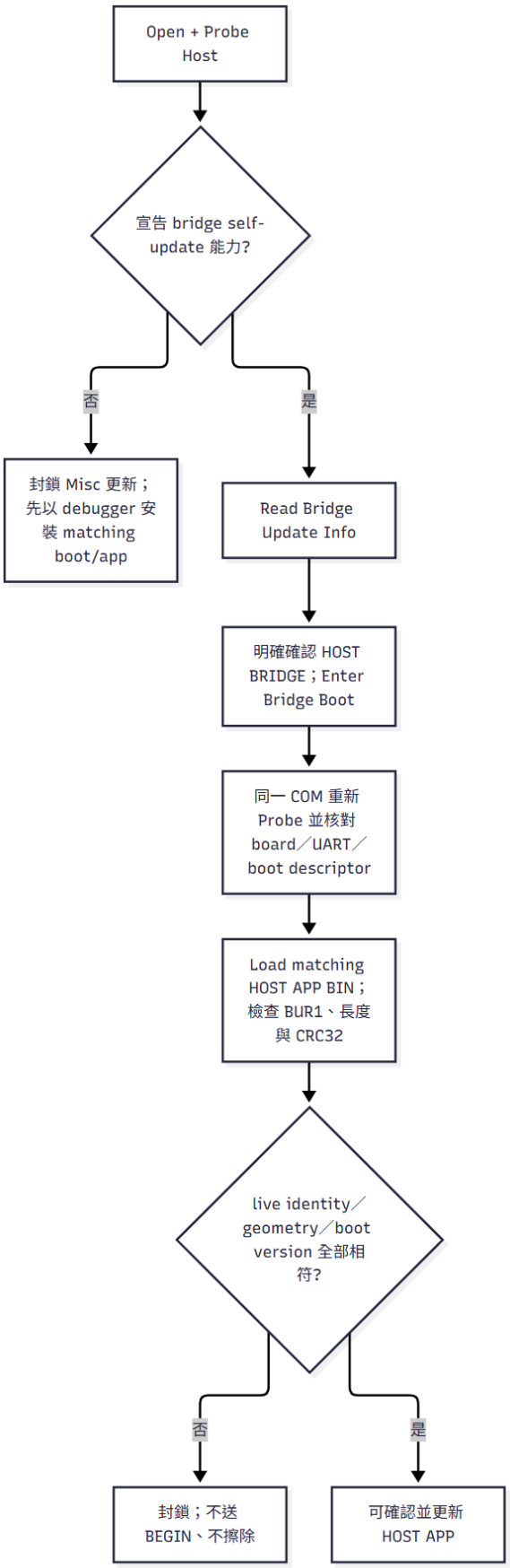
COM 清單的 USB friendly name 用於辨識轉接器。 `Open + Probe Host` 取得實際 bridge board 的 board ID、韌體版本、能力與可選 port。 UI 依 Host Link / backend 回報的能力顯示功能與 port。

CAN / I2C 模式的 BOOT / INSTALLED APP 卡來自外部 device 的 live INFO；LOADED BIN 是離線檔案。Custom Transfer 不提供 boot/app metadata，所以顯示 N/A。G16 範例回報的版本是常數，使用黃色；其 erase/write/jump 仍封鎖。

Misc 的 BOOT / APP 卡改為 host bridge 自己的資訊，且目的地卡明確顯示 HOST BRIDGE。離線 BIN 仍獨立。CRC 是完整性檢查，不是簽章、授權或防回滾。

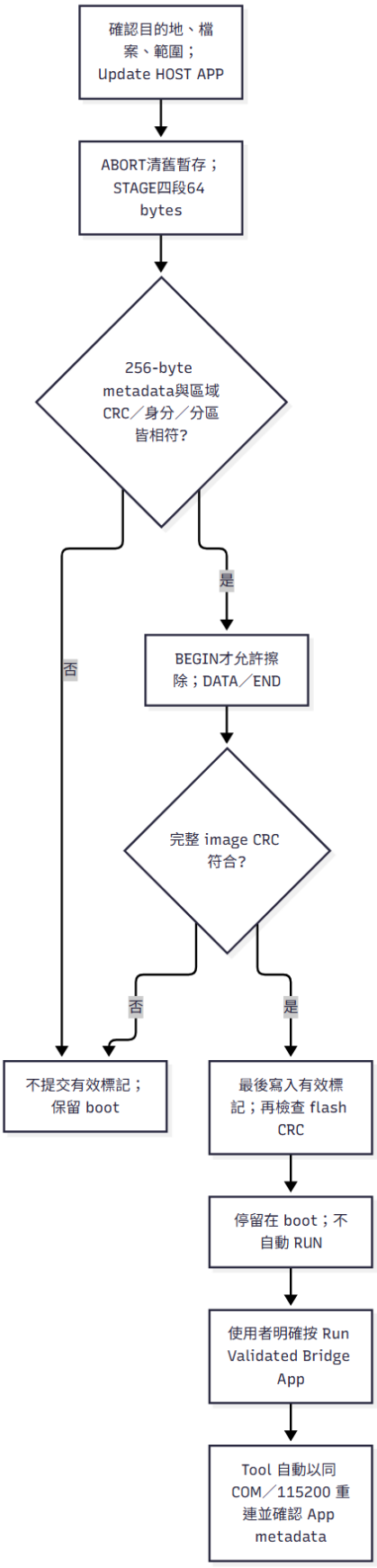
3. Misc：PC 透過 UART 更新 Host App

Misc 透過 PC □ host 的 UART 更新 Host App。 UART baud rate 與 CAN bitrate 分別設定。



3.1 寫入與啟動分開

寫入與執行是不同動作，成功寫入後不會直接跳轉。



開始寫入前，確認視窗會列出目的地 HOST BRIDGE、COM、MCU part、檔案、app start 與大小。預設選 No。TAB 切換、載入檔案、Read Info 都不會擦寫。STAGE只有RAM暫存，缺段、衝突、總長 / CRC / 身分不符均不擦Flash。Read Info在有有效app時以相同READ_METADATA command、offset0/64/128/192讀回完整metadata並核對descriptor。更新完成也分段read-back核對；成功後仍需明確RUN。Close COM可取消，tool嘗試ABORT清除staging再關閉；BEGIN前取消不改舊app，BEGIN後不保證舊app仍在。

未宣告 self-update 能力的 bridge 不開放 Misc 寫入，不推測位址或接受 raw BIN。

更新後確認、Compare Metadata、分階段進度、Cancel 與自動報告，集中於 更新確認與驗證紀錄。

4. F24 (MCU CAN host) 分區與建置

實際 MCU：R7F124FLJ3xFB，256 KiB code flash。Boot 為 20 KiB，App start 為 0x5000。最後 1 KiB 保留，不列入更新範圍。

區域	位址 / 容量	規則
Boot	0x00000..0x04FFF，20 KiB	Misc 絕不更新 boot
Host app	0x05000..0x3FBFF，235 KiB	動態 image length，4-byte 對齊
保留區	0x3FC00..0x3FFFF，1 KiB	包含 debug monitor，不可由 boot 擦寫
IRQ 跳轉表	app offset 0x000..0x0FF	64 組四位元組 BR 入口
BUR1 metadata區	app offset 0x100..0x1FF	256 bytes：core128 / reserved128，區分HOST APP與外部 device image
App code	app offset 0x200 起	由 linker 決定實際結尾
CRC32	實際 image 最後 4 bytes	對前面全部 bytes 計算，little-endian 儲存

區域容量256、有效core長度128、UART payload128、metadata chunk64是四個獨立尺寸。8-bit長度欄位表示core128，分段offset / total則使用16-bit。最小image為516 bytes，避免最後CRC覆蓋metadata。Reserved 欄位依契約驗證。

單一分區來源為 `firmware/f24_bridge_common/f24_bridge_layout.h`。唯一兩份 CS+ 產品專案是 `RL78_F24_Bridge_boot` 與 `RL78_F24_Bridge_app`；兩者引用 `firmware/f24_board` 的硬體來源。共用目錄沒有 MTPJ / 可燒錄產物；不要在 projection 內另行修改 Smart Configurator 輸出。

參考專案硬體設定重新產生後，以 `scripts/prepare_f24_bridge_projects.ps1 -Kind app` 更新 app projection / 專案；它保留 app-local mapping，不更新 boot。只有明確修改 boot 自己的設定才使用 `-Kind boot`；省略 Kind 仍表示同時產生兩者，不用於日常 app-only 更新。CS+ app pre-build 會產表，post-build 會檢查並封裝。

App 的 `DefaultBuild/*_bridge_update.bin` 是 Misc 要載入的檔案；`*_flash.mot` 是已加入 metadata / CRC 且排除低位向量、option、debug 區的初次 app 燒錄產物。不要把原始 app `.mot` / `.abs` 當成獨立可啟動韌體：device file 會產生未綁定的低位向量 placeholder。

首次安裝 Boot 使用 debugger / programmer，可採下列方式：

- 同時安裝 Boot `.mot` 與 matching App `_flash.mot`。
- 只安裝 Boot，沒有有效 App 時停留 recovery；再於 Misc / 115200 載入 matching `_bridge_update.bin`，驗證及更新後執行 `Run Validated Bridge App`。

Misc 僅更新 Host App。 調整分區時，Boot 與 App 的配置須一致，並以燒錄器重新安裝對應 Boot。

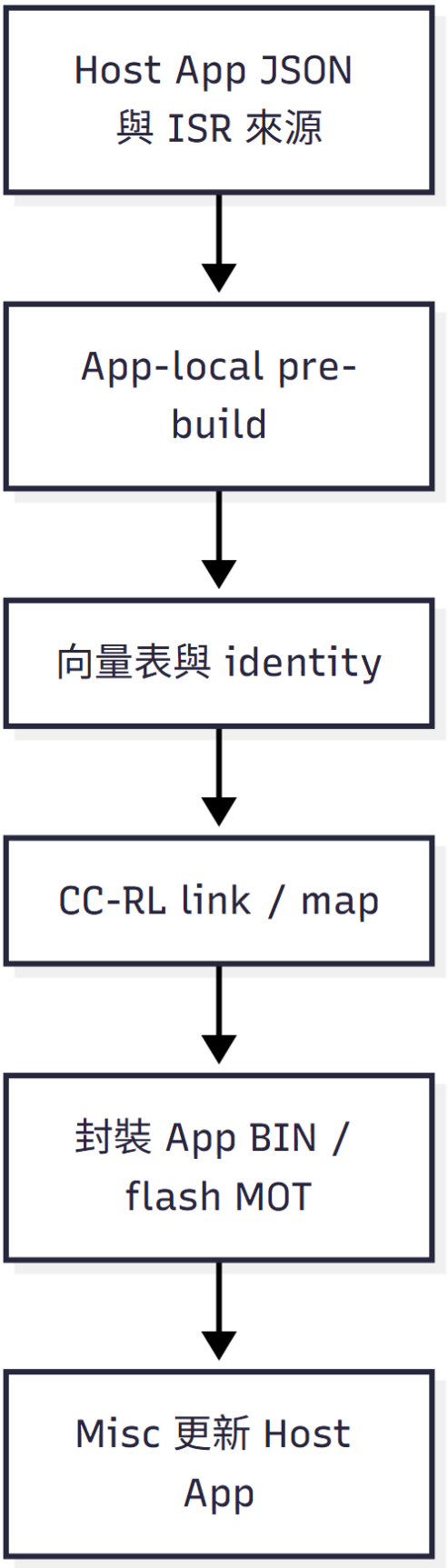
F24 (MCU CAN host)：App 工程

本節供需要閱讀或調整 host firmware 的開發者參考。

來源目錄	內容
`firmware/RL78_F24_Bridge_boot`	Host Boot、UART recovery
`firmware/RL78_F24_Bridge_app`	Host App、Host Link dispatch、bus service
`firmware/f24_board`	共用 board / driver / SC
`firmware/f24_bridge_common`	Host App 分區與自更新契約

Host App 擁有自己的 `app_image.json` 與 `app_irq_handlers.json`。 Pre-build 產生 `bridge_vectors.asm` / `f24_app_identity.generated.h`，post-build 依 map 封裝更新映像。

F24 (MCU CAN host)：App 工程



App-only 建置

1. 建立自己的 App 工程，保留 MTPJ、startup、兩份 JSON 與來源登錄。
2. Pre-build 使用 `prepare_f24_app_irq.ps1 -AppProject "."`。
3. Post-build 使用 `package_f24_bridge.ps1 -Kind app -AppProject "."`。
4. 在 Tool root 執行對應 App 的建置：

```
.\scripts\build_f24_bridge.ps1 -Kind app -AppProject ".\ProjectA"
```

封裝輸出 `<  >_bridge_update.bin` 與 `<  >_flash.mot`。 產表與封裝會核對 App mapping、BR 目的地及 RETI / RETB。 更新操作集中於 Misc / UART。

Smart Configurator

SC 設定為 `firmware/f24_board/f24_board.scfg`。 搬移或重新產生 SC 後，以 `prepare_f24_bridge_projects.ps1 -Kind app` 整理 App 的相對路徑與來源登錄，再建置 App。 Host Boot 的硬體設定調整時，另以 `-Kind boot` 整理及建置。

5. MCU 程式與硬體注意事項

Boot UART 使用 HOCO40 與 115200；app 沿用已確認的 X1 20 MHz、PLL80、CPU/CAN40 設定。CAN TX/RX=P10/P11，STB=P12；UART TX/RX=P15/P16；IICA0=P62/P63。

RFD Type02 與執行中的 flash driver 都必須從 RAM 執行；programming mode 期間禁止 code flash 的程式、常數或 IRQ 取用。程式已使用 RAM relocation、關閉 IRQ、有限次數 sequencer polling，回到 non-programmable mode 才回傳；無法安全恢復時由 RAM 重置並留在 boot。這些是 source / linker 檢查，不是實際電氣與 flash 時序驗證。

明確配置 4 KiB stack，避免自動 stack 初始化覆蓋 FF700 的 recovery cookie 與 FF710 的 IRQ dispatch selector。BEGIN 僅可擦 app 區；DATA 連續且四位元組對齊。有效 manifest magic 最後才寫入；完整 image CRC 不符、傳輸未完成、重上電遇到無效 image 都不能跳 app。SW1 可要求 boot recovery。

0x7E 的 BRK 是軟體中斷，需要 RETB 而非一般 IRQ 的 RETI；boot 不覆寫 / 轉送該 physical vector，保留給 debugger / device file。App 表仍保留對應 slot。CS+ halt、continue、stop 與 E2 lite 連線仍須實際測試。

CAN analyzer self test 的資料為 standard ID 0x123、DLC8、`11 22 33 44 55 66 77 XX`，每秒一次；每次開始 XX 從 00 遞增。須搭配 CAN analyzer 觀察實體 bus，UART TX log 不能證明實際波形正確。

接線時依各板 transceiver 的規格核對供電及 standby，並確認 CAN H/L 終端、共地與 analyzer bitrate。

6. 驗證紀錄

軟體檢查涵蓋 map / 向量 / RAM / 封裝、parser 與更新狀態機。 實板檢查涵蓋 UART 傳輸、Flash 擦寫、reset 後 App、IRQ、取消與斷電復原。 版本與量測結果收於 交付紀錄。

更新政策與通訊速度

BIN 的更新目的地、允許介面、Boot / IRQ ABI 與 Host Link 速度，集中於 ISP Tool 的更新政策章節。 閱讀完整章節